Firmware Reference

Main ROM IC519 - Disassembly, Command Set and Data Formats

Reverse-Engineering Supplement to the Operation and Service Manuals
Document VS-2724A-90F01
Derived from ROM images 2724ASTD.bin and 2724ACPR.bin
Revision 1.1
Jared Cabot • jetstreamtechnology@protonmail.com
Disassembly and analysis by Claude (Anthropic)

Scope

This supplement documents the firmware held in IC519, the 8K x 8 main ROM of the Valhalla Scientific Model 2724A and 2724CPR Programmable Precision Resistance Standard. It covers the processor and memory map, the IEEE-488 command interpreter, the display and message subsystem, the non-volatile data format and the calibration state machine.

Its purpose is to record what the firmware actually does, and in particular to record the behaviour that the Operation Manual and Service Manual do not describe. Where this document and the factory manuals disagree, the disagreement is stated explicitly rather than resolved silently.

This is not a repair manual. Circuit description, adjustment procedures and parts lists remain the province of the Service Manual, which this document supplements and does not replace. Nothing here describes the Option TL-1 IEEE board firmware, which is not held in IC519.

Provenance

Every statement in this document derives from static analysis of two ROM images: 2724ASTD.bin (8192 bytes, raw binary) and 2724ACPR.hex (Intel HEX, 8192 bytes decoded). Both were disassembled for the Motorola 6809 and traced from their hardware vectors.

Two independent checks support the reading. First, both images pass the firmware's own power-on ROM test: the sum of every 16-bit word from $E002 to $FFFF equals the word stored at $E000 in both cases, which confirms the images are complete, correctly ordered and uncorrupted. Second, all seven live interrupt vectors in both images point into the ROM, which confirms the load address.

Hardware facts - the processor type, the memory devices and the interrupt assignments - are taken from the Service Manual parts list and Section 9-3, and are cited as such. Everything else is read from the code.

No instrument was available. Nothing in this document has been confirmed by operating a 2724A. Chapter 9 records exactly what was and was not checked.

Applicability

Two firmware builds are described:

Standard, Valhalla part 03-50019, ROM checksum $F688. CPR, Valhalla part 03-50126, ROM checksum $A4B9. The checksum is the word at $E000 and is the quickest way to identify a device.

The two builds share only 15.2% of their bytes at the same address. Addresses given in this document therefore always state which build they refer to. A reader working on a device whose checksum matches neither value should treat every address here as unverified.

Conventions

Hexadecimal values carry a $ prefix, addresses and register contents alike. Binary values carry a % prefix. All addresses are CPU addresses in the 6809's 64K space unless stated otherwise.

Text in the monospaced face is either something sent to or received from the instrument, or a symbol taken from the firmware. Angle brackets enclose a placeholder: <n> is a single decimal digit.

'Documented' means the behaviour appears in the 2724A Operation Manual or Service Manual. 'Undocumented' means it appears in neither. This distinction is the point of the document and is applied strictly.

Disclaimer

This is not a Valhalla Scientific publication and carries no endorsement from that company or its successors. It was produced by disassembling firmware for the purpose of maintaining and repairing obsolete test equipment.

The undocumented commands recorded here are undocumented for reasons that are not recorded. Several of them write to non-volatile memory or drive the calibration state machine. Using them on a working, calibrated instrument may destroy its calibration. Read Chapter 9 before acting on anything in this document.

Contents

1.):] out, cur = [], 0 for mm in _re.finditer(r'm\.chapter\(\s*(\d+),\s* **1-1**

1. General Information **1-1**

The Instrument and Its Firmware 1-1

Identifying a ROM 1-1

Summary of Undocumented Findings 1-2

2. Processor and Memory **2-1**

Processor 2-1

Interrupt Vectors 2-1

Memory Map 2-2

Peripheral Addresses 2-2

Instrument I/O Ports 2-3

Read/Write Memory Layout 2-4

3. Display Subsystem **3-1**

Character Set 3-1

Message Format 3-1

Keyboard Scanning 3-2

Key Matrix Map 3-2

Matrix Differences in the CPR Build 3-3

Greeting Selection 3-3

The Remaining Two Straps 3-4

4. Message Catalogue **4-1**

Messages Present in Both Builds 4-1

Messages Absent from the Manuals 4-2

Discrepancies with Table 3-2 4-3

5. Remote Operation **5-1**

Interface Hardware 5-1

Character Reception and the Input Buffer 5-1

The Command Interpreter 5-2

Unrecognised Commands 5-2

6. Programming Reference **6-1**

The Complete Command Set 6-1

Argument Ranges Enforced by the Firmware 6-2

Numeric Entry 6-3

Commands Absent from the Manuals 6-3

Contents (cont'd)

7. Command Reference **7-1**
Introduction 7-1
Notation 7-1
Command Summary 7-1
Command Entries 7-2

8. Non-Volatile Data **8-1**
NOVRAM Organisation 8-1
Block Layout and Checksums 8-1
Store and Recall 8-2
Corruption Behaviour 8-2

9. Calibration and Integrity Checks **9-1**
The Calibration Step Counter 9-1
Driving Calibration Remotely 9-1
The Banner Integrity Check 9-2

10. Firmware Differences **10-1**
Common Regions 10-1
Functional Differences 10-2

11. Verification Record **11-1**
Confirmed by Computation 11-1
Not Verified 11-2
How to Reproduce 11-2

1

General Information

The Instrument and Its Firmware

The Valhalla Scientific Model 2724A is a programmable precision resistance standard. It does not contain a switched resistor network; it synthesises a resistance actively, by measuring the current forced through an internal reference resistor and driving its output terminals to the voltage that a resistor of the requested value would develop. The Operation Manual calls this the active ohm technique. Option CPR adds a set of fixed cardinal-point resistors and a second operating mode.

All of the instrument's behaviour above the analogue sections is software. A single 8K x 8 EPROM at IC519 holds the display multiplexer, the keyboard scanner, the IEEE-488 command interpreter, the calibration state machine and the non-volatile data manager. Two builds of that ROM exist, one for the standard instrument and one for the CPR option.

The Service Manual gives the processor as an MC68A09P at IC518 and describes the memory complement in Section 9-3-6-5. It does not document the firmware. This supplement does.

Identifying a ROM

The firmware verifies itself at every power-on by summing all 8190 bytes above $E001 as 16-bit words and comparing the result with the word stored at $E000. That stored word is therefore a reliable build identifier and can be read directly from a device image without disassembly.

Table 1-1. ROM Build Identification

Build	Valhalla part	Checksum at $E000	Size	Image analysed
Standard	03-50019	$F688	8192 bytes	2724ASTD.bin
CPR	03-50126	$A4B9	8192 bytes	2724ACPR.hex

Note



Both images analysed for this document pass their own checksum test. A device whose computed sum does not equal the word at $E000 will hang at reset with all display decimal points lit, which the Operation Manual lists in Table 3-2 as ROM failure.

Summary of Undocumented Findings

The following are present in both firmware builds and appear in neither factory manual. Each is developed in the chapter cited.

Table 1-2. Undocumented Behaviour at a Glance

Finding	Chapter
Bus command `SM<n>` stores the present value into user memory	4, 11
Bus command `RM<n>` recalls user memory	4, 11
Bus command `RL` recalls the last entered value	4, 11
Bus command `N` advances the calibration sequence	4, 8, 11
Message `IEEE NOT FITTED`	6
Messages `CAL ABORTED`, `FAST DATA OK`, `NO FAST DATA`, `UNDERCURRENT`	6
A banner-comparison integrity check with an obscene failure message	8
Six characters silently discarded by the parser	3
The firmware spells the A-to-D fault `A-D TIMEOUT`, the manual spells it `ATOD TIMEOUT`	6

2

Processor and Memory

Processor

IC518 is an MC68A09P, the 1.5 MHz grade of the Motorola 6809. The Service Manual parts list gives the part directly; the firmware confirms it, because the code uses instructions and addressing modes that exist on no other 8-bit Motorola part - `LEAX/LEAY` with program-counter-relative offsets, the `U` stack, `TFR` and `EXG` register pairs, and a fourth interrupt, `FIRQ`, which is unique to the 6809 family.

The direct page register is loaded with $C0 during reset and is never changed afterwards. Every direct-page reference in the firmware therefore addresses $C000 to $C0FF, and this document writes such operands as `<$nn` with their full address given where it matters.

Interrupt Vectors

The Service Manual, Section 9-3-6-3, assigns the four interrupts. The firmware vectors agree with that assignment and are listed below for both builds.

Table 2-1. Interrupt and Reset Vectors

Vector	Name	Standard	CPR	Service
$FFF0	(reserved)	$0000	$0000	Unused
$FFF2	SWI3	$EBB0	$EE29	Internal
$FFF4	SWI2	$EB85	$EDFE	Internal
$FFF6	FIRQ	$E860	$EAAF	Display digit position due for update
$FFF8	IRQ	$EFF1	$F2B5	IEEE-488 bus activity
$FFFA	SWI	$EF2C	$F1F0	Command line complete; run the interpreter
$FFFC	NMI	$EBE3	$EE5C	New A-to-D measurement available
$FFFE	RESET	$F9D8	$FD51	Power-on and remote reset

Note



The `SWI` vector is not an error trap in this design. The IEEE-488 receive interrupt executes a `SWI` once a complete command line has been buffered, and the interpreter runs in the software interrupt handler. See Chapter 5.

Memory Map

Address decoding uses only A9 to A15, so the map has a granularity of 512 bytes and every device appears repeated through its block. The addresses given below are those the firmware actually uses.

Table 2-2. Memory Map

Address range	Device	Size	Notes
$0000 - $0FFF	Output latches (IC512-IC517, IC526)	-	Write-only; selected on 512-byte boundaries
$1000 - $1007	IEEE-488 interface (Option TL-1)	8 bytes	Talker/listener controller; absent when the option is not fitted
$2000	NOVRAM STORE strobe	-	Read to copy RAM into NOVRAM
$4000	NOVRAM RECALL strobe	-	Read to copy NOVRAM into RAM
$A000 - $A0B3	NOVRAM (IC522 and IC523)	180 nibble pairs	IC522 supplies D0-D3, IC523 supplies D4-D7
$C000 - $C3FF	Static RAM (IC520, IC521)	1024 bytes	Direct page is $C000-$C0FF
$E000 - $FFFF	Main ROM (IC519)	8192 bytes	Checksum word at $E000; vectors at $FFF0

Peripheral Addresses

The IEEE-488 controller occupies eight consecutive locations. The firmware's use of each is given below; register names follow the usual talker/listener controller convention rather than any datasheet, because the device type is not named in the parts list.

Table 2-3. IEEE-488 Controller Registers as Used by the Firmware

Address	Access	Firmware use
$1000	R/W	Interrupt status; also written with the enable shadow held at `<$59`
$1001	R	Secondary status - bus command and addressing events
$1002	W	Written once during initialisation
$1003	W	Auxiliary command register; $20 and $10 written in the receive path
$1004	R/W	Address register
$1005	R/W	Serial poll / status byte
$1006	W	Parallel poll response
$1007	R/W	Data in and data out

Instrument I/O Ports

Table 2-2 describes the latch block as a whole. Within it the firmware uses nine specific addresses, and the purpose of each can be recovered from the code that touches it. None of these assignments appears in the manuals.

Table 2-4. Input and Output Ports Used by the Firmware

Address	Access	Width	Function
$0000	R	8	Front-panel column read. Bit 7 marks the end of a scan pass
$0200	W	8	Cleared once during reset
$03FF	W	16	Display segment and digit drive, written by the FIRQ handler
$0400	W	8	Written with $40 once during reset
$07FF	W	16	Main relay and range data, written with interrupts masked
$0A00	W	8	Further relay data, written immediately after $07FF in the same masked region
$0C00	W	8	Mode latch. Bit 7 is set when the instrument is in active rather than CPR mode
$0E00	W	8	Mode latch. Bit 7 is set when the instrument is in slow rather than fast mode
$13FF	R	16	A-to-D converter result, read by the NMI handler

Two of these deserve comment. The pair at $07FF and $0A00 is always written together with interrupts masked by `ORCC #$50`, which is what one would expect of a relay tree that must not be left half-updated. And the two mode latches carry their flag inverted: bit 7 is set for the *power-up* condition of each mode pair, so an unprogrammed instrument presents active mode and slow mode.

Read/Write Memory Layout

The reset routine tests all 1024 bytes of RAM with three patterns before use. The allocations below are those that can be identified from the code with confidence; the remainder of the direct page holds working variables whose purpose was not established.

Table 2-5. Identified RAM Allocations

Address	Bytes	Contents
$C012	1	Calibration step counter; zero when not calibrating
$C017	1	Switch and link row, latched at display scan position 8
$C04F	1	SRQ mask, set by `Q`
$C050	1	Output delimiter code, set by `E`
$C051	1	Parallel poll response line, set by `P`
$C052	1	Two/four-wire selection, set by `T`
$C053	1	Fast/slow selection, set by `M`
$C054	1	CPR/active selection, set by `C` (CPR build only)
$C05A	2	Input buffer write pointer
$C05D	2	Interpreter read pointer
$C066	1	Memory slot number for `SM` and `RM`
$C069	8	Present output value
$C071	8	Value being entered
$C134	100	IEEE-488 input buffer; upper limit $C198
$C1CA	8	Last entered value, for `RL`
$C23F	-	Stack top
$C300	88	Calibration data
$C358	2	Calibration data checksum
$C35A	2	User configuration, including IEEE address
$C35C	80	Ten user memories of 8 bytes each
$C3AC	2	User data checksum

3

Display Subsystem

Character Set

The display is a 16-digit, 14-segment vacuum fluorescent panel, multiplexed at 2.66 kHz. The firmware does not store text as ASCII. It stores an index into a 43-entry table at $E058, and that table maps each index to the ASCII code the segment decoder expects. The table is byte-identical in both builds.

Table 3-1. Display Character Table at $E058

Index range	Characters
$00 $01 $02 $03 $04 $05 $06 $07 $08 $09 $0A	0 1 2 3 4 5 6 7 8 9 *
$0B $0C $0D $0E $0F $10 $11 $12 $13 $14 $15	A C D E B F G H I J
$16 $17 $18 $19 $1A $1B $1C $1D $1E $1F $20	K L M N O P Q R S T U
$21 $22 $23 $24 $25 $26 $27 $28 $29 $2A	V W X Y Z - / o u m

The ordering repays attention. Indices $00 to $09 are the digits, so a numeric value can be displayed without translation. Index $0F is the space. The letters are not in alphabetical order: A C D E come before the space and B follows it, which suggests the table grew as messages were written rather than being laid out in advance. Only 43 of the 256 possible codes are defined; a message byte above $2A has no character and will display unpredictably.

Message Format

Every message is exactly 15 bytes of character-table indices, stored without a terminator or a length byte. Messages are packed end to end into blocks, and a message is selected by multiplying its index within the block by 15 and adding the block base. The code that does this is explicit:

```
LEAX  block,PCR      ; base of the message block
DECA                 ; message number, 1-based
LDB   #$0F           ; 15 bytes per record
MUL
ABX                  ; X = block + (n-1) * 15
LBSR  display        ; $E9E6 standard, $EC55 CPR
```

The display routine takes the pointer in X and writes 15 characters. The sixteenth digit is not written from the message and carries annunciator state.

Keyboard Scanning

The keyboard shares the FIRQ handler with the display. Each interrupt writes the next display word to $03FF and then reads $0000, so the column lines are sampled against whichever digit line is currently driven. A scan pass ends when bit 7 of the read comes back set.

A pressed node is latched as a column mask at `<$1F` and a row number at `<$20`. The handler compares each new sample against those two bytes and acts only on a change, which is the debounce. Both are cleared on release, so a held key produces exactly one action.

The decoder at $F77D (CPR; $F432 standard) turns the pair into a key number. It shifts the column mask right twice, finds the lowest set bit, and forms an index of four times the row plus that bit position. If more than one bit remains set the routine returns $FF and the press is discarded, so simultaneous presses are rejected rather than resolved.

The index selects a two-byte entry from the table at $F79C (CPR; $F43E standard). The first byte is the key number used to select a handler from the key dispatch table of Chapter 6; the second is a sub-code left at `<$79` for that handler to read. This is how one handler serves several keys.

Key Matrix Map

The matrix is nine rows of four columns. Entries below are the key number and, where it is significant, the sub-code. A dash marks a node with no key.

Table 3-3. Key Matrix, Standard Build

Row	Column bit 2	Column bit 3	Column bit 4	Column bit 5
1	8	9	10	-
2	12	11.2	11.1	11.0
3	5.9	5.6	5.3	6
4	5.8	5.5	5.2	5.0
5	5.7	5.4	5.1	7
6	4	3	1	2
7	-	-	-	0
8	-	-	-	-
9	14.1	14	13	13.1

Key 5 is numeric entry and its sub-code is the digit value, which is why rows three to five read as a keypad: 9 6 3, 8 5 2 0, 7 4 1. Key 11 appears three times with sub-codes 2, 1 and 0; these are the unit keys, and key 11 is the same handler the bus command `N` reaches, which is what makes calibration remotely drivable. See Chapter 9.

Note



Row 1, column bit 5 has no table entry, but the FIRQ handler tests for that node explicitly before consulting the table and branches into the IEEE-488 code instead. It is the local/remote key, and it is handled outside the normal dispatch so that it works while the instrument is addressed.

Matrix Differences in the CPR Build

Eight of the nine rows are identical between the builds. Row nine differs, and it is where the CPR instrument's two extra keys live.

Table 3-4. Row Nine in Both Builds

Build	Column bit 2	Column bit 3	Column bit 4	Column bit 5
Standard	14.1	14	13	13.1
CPR	14.1	16	15	13.1

Key 15 in the CPR build toggles the byte at `<$54` and displays `SEL CPR MODE` or `SEL ACTIVE` accordingly. That is the same variable the bus command `C` writes, so the key and the command are two routes to one setting. Key 16 is refused unless `<$54` is already set, and complements `<$7A`.

The standard build maps those two matrix nodes to keys 14 and 13, duplicating the functions of their neighbours. A standard instrument therefore has no dead nodes where the CPR keys would be.

Greeting Selection

The Operation Manual notes that instruments shipped outside the United States may greet the operator in another language, but does not say how the language is chosen. Drawing 2724-071, the display and keyboard schematic, does: note 4 on that sheet gives the selection table directly. The mechanism is a hardware option, not a firmware variant, and both builds contain all four greetings.

Table 3-2. Greeting Selection, from Drawing 2724-071 Note 4

Diodes fitted	Note 4 text	$C017 bits 3,2	Index	Firmware text
None	HI-THERE	%11	0	`HI-THERE`
D701	HELLO	%10	1	`HELLO`
D702	GUTEN TAG	%01	2	`GUTEN TAG`
All	BON JOUR	%00	3	`BONJOUR`

D701 to D704 are four optional 1N4148 diodes on the display board. Their cathodes are commoned; their anodes go to the four lines K1 to K4, each held up by a 1 k ohm resistor, R703 to R706, to +5 V(E) and each leaving the board on SK2-57 to SK2-60. Fitting a diode pulls its line low. The same four lines carry the `2 WIRE`, `4 WIRE`, `SLOW MODE` and `FAST MODE` keys at other scan positions, so the straps cost no extra wiring.

The firmware reads that row at display scan position 8 and stores it at $C017. Only positions 0 to 7 and 9 are treated as keys; position 8 is reserved for switches and straps. The greeting index is then formed from bits 3 and 2:

```
LDA   <$17          ; switch and strap row
ORA   #$F3          ; keep bits 3 and 2
COMA                ; active low - a fitted diode is a 1
LSRA
LSRA                ; index 0 to 3
LDB   #$0F
MUL
ABX                 ; X = greeting block + index * 15
```

The COMA is what makes the table read the way it does. An unfitted position sits high, so no diodes gives an index of zero and the first greeting. Evaluating the arithmetic for all four strap combinations reproduces note 4 exactly, which confirms both the reading of the code and the bit assignment: D701 drives bit 2, D702 drives bit 3.

Note



Note 4 writes the fourth greeting as two words, BON JOUR. The firmware stores it as one, BONJOUR. The first greeting differs the same way: hyphenated in the firmware, unhyphenated in the Operation Manual.

The Remaining Two Straps

Note 4 accounts for D701 and D702 only. D703 and D704 are fitted to the same pull-up network on K3 and K4 and no drawing note gives them a function.

The firmware tests two further bits of $C017, bit 0 and bit 1, neither of which belongs to the greeting. Bit 1 sets a flag at $C089 once per pass of the main loop. Bit 0 is more interesting: when it is set the main loop clears the output-enable flag at $C084, refreshes the output latches and returns to the top without executing the normal output path, which is the behaviour of an output inhibit.

Whether D703 and D704 drive these two bits was not established. The wiring from K3 and K4 through the connector to the keyboard buffer IC510 would settle it, and that runs onto drawing 2724-070 sheet 6, which was not traced. Note also that the sense is opposite to the greeting bits - these two are tested directly rather than complemented, so on these lines a fitted diode reads as zero - which is consistent with them being operational inputs rather than shipping straps.

4

Message Catalogue

Messages Present in Both Builds

The catalogue below was produced by decoding every run of valid character-table indices in both images through the table at $E058 and keeping those that form 15-character records. Addresses are given for both builds. Text is reproduced exactly, including the spacing that centres each message on the display.

Table 4-1. Fault and Status Messages

Message	Std	CPR	Meaning	In manuals
`TURN CAL SW OFF`	$E083	$E083	Calibration switch still on at the end of a calibration	Yes
`CAL DATA OK`	$E092	$E092	Power-on self-test result	Yes
`CAL ABORTED`	$E3C9	$E503	Calibration sequence abandoned	No
`FAST DATA OK`	$E3D8	$E512	Fast-mode calibration data accepted	No
`NO FAST DATA`	$E3E7	$E521	No fast-mode calibration data present	No
`FUCK FUCK FUCK`	$E405	$E53F	Banner integrity check failed - see Chapter 9	No
`CLOCK FAILURE`	$EAFE	$ED77	Faulty clock circuit	Yes
`MATH OVERFLOW`	$EB0D	$ED86	Arithmetic overflow	Yes
`CAL DATA BAD`	$EB1C	$ED95	Calibration data corrupted	Yes
`MEMORY DATA BAD`	$EB2B	$EDA4	User memory data corrupted	Yes
`OUT OF CAL`	$EB3A	$EDB3	A component is out of tolerance	Yes
`SETTLING`	$EB49	$EDC2	Output has not yet settled	Yes
`CANNOT CONTROL`	$EB58	$EDD1	Output cannot reach the requested value	Yes
`IEEE NOT FITTED`	$EB67	$EDE0	IEEE-488 option not installed	No
`A-D TIMEOUT`	$EB76	$EDEF	A-to-D conversion timed out	Partly

Table 4-2. Prompt and Greeting Messages

Message	Std	CPR	Meaning	In manuals
STORE MEMORY	$F25A	$F535	Prompt: awaiting a memory number to store	No
RECALL MEMORY	$F29D	$F57A	Prompt: awaiting a memory number to recall	No
UNDERCURRENT	$F356	$F668	Test current below the usable minimum	No
SEL SLOW MODE	$F38A	$F69C	Prompt: slow mode selected	No
SEL FAST MODE	$F399	$F6AB	Prompt: fast mode selected	No
SEL FOUR-WIRE	$F3A8	$F6BA	Prompt: four-wire selected	No
SEL TWO-WIRE	$F3B7	$F6C9	Prompt: two-wire selected	No
SEL CPR MODE	$F3C6	$F6D8	Prompt: CPR mode selected	No
SEL ACTIVE	$F3D5	$F6E7	Prompt: active mode selected	No
HI-THERE	$F982	$FCFB	Power-on greeting, link selected	Yes
HELLO	$F991	$FD0A	Power-on greeting, link selected	Yes
GUTEN TAG	$F9A0	$FD19	Power-on greeting, link selected	Yes
BONJOUR	$F9AF	$FD28	Power-on greeting, link selected	Yes

Messages Absent from the Manuals

Ten of the twenty-eight messages appear in neither factory manual. Four of them a user may see in ordinary operation.

- IEEE NOT FITTED - shown when a bus function is attempted on an instrument without Option TL-1. Table 3-2 of the Operation Manual lists no such message.
- UNDERCURRENT - the counterpart of the overcurrent indication. The Operation Manual mentions an undercurrent state in the output format of Section 4.5, where a U appears in the status field, but never lists the display message.
- STORE MEMORY and RECALL MEMORY - prompts shown while the instrument waits for a memory digit.
- The six SEL prompts, shown as each mode is selected.

The remaining four - CAL ABORTED , FAST DATA OK , NO FAST DATA and the banner-check failure message - belong to the calibration sequence and are covered in Chapter 9.

Discrepancies with Table 3-2

Two differences between the firmware and Table 3-2 of the Operation Manual are worth recording, because a technician matching a display against the manual will not find them.

Table 4-3. Firmware Text Against Manual Text

Manual, Table 3-2	Firmware	Comment
`ATOD TIMEOUT`	`A-D TIMEOUT`	Different spelling; same fault
`HI THERE`	`HI-THERE`	Greeting is hyphenated in the firmware
-	`IEEE NOT FITTED`	Message absent from the manual

Note



Table 3-2 introduces its list with the words `Some of the messages`, so the manual does not claim to be complete. It is nonetheless the only list a user has, and it omits a message that a unit without the IEEE option can display.

5

Remote Operation

Interface Hardware

IEEE-488 is Option TL-1 and lives on a separate board, 2724-602. The main ROM contains the whole of the command interpreter regardless of whether the option is fitted; only the eight interface registers at $1000 are absent from an instrument without the board. The firmware detects this and can display `IEEE NOT FITTED`, a message that appears in neither manual.

Character Reception and the Input Buffer

Received characters are handled entirely in the IRQ handler. Each byte is read from $1007, bit 7 is masked off, and the result is appended to a 100-byte buffer running from $C134 to $C197. Nothing is interpreted at this stage.

Three conditions end a line:

- EOI asserted with the character, detected from the status register.
- A carriage return, `$0D`.
- The buffer reaching its limit at $C198.

On any of these the handler clears the receive enable, writes a terminating zero at the buffer position, and executes `SWI`. The interpreter therefore runs in the software interrupt handler, at foreground priority rather than inside the receive interrupt.

One character is discarded before it ever reaches the buffer: a space, `$20`, is dropped in the interrupt handler. A command line may therefore be padded with spaces anywhere without effect, including in the middle of a number.

Caution



Line feed is not a terminator. A controller that ends its output with carriage return and line feed will leave the line feed at the head of the next buffer, where the interpreter discards it. A controller that sends line feed alone will never terminate a line, and the instrument will accept characters until the buffer fills at 100 bytes.

The Command Interpreter

The interpreter is a straightforward table walk. It takes the next character from the buffer and searches a dispatch table of four-byte entries: one ASCII character followed by a three-byte long branch to the handler. The table ends with a zero byte. On a match the interpreter jumps directly into the table entry, so the long branch itself performs the dispatch.

```
entry:   FCB   'x'          ; the command character
         LBRA  handler      ; three bytes, executed in place
...
         FCB   $00          ; end of table
```

If the character is not in the dispatch table, a second and much shorter table is searched. Characters found there are discarded silently and the interpreter moves on. This second table is the reason punctuation may be sprinkled through a command line without causing an error.

Table 5-1. Characters Discarded Without Error

Code	Character	Name
$2C	,	comma
$3B	;	semicolon
$3A	:	colon
$2D	-	minus
$2B	+	plus
$0A	-	line feed

Note



The minus and plus signs are in this list. At the top level of a command line a leading sign is therefore ignored rather than negating anything. Signs are recognised only inside an exponent, where the numeric parser handles them separately. The skip list is identical in both builds.

Unrecognised Commands

A character in neither table is an error. The firmware tests bit 1 of the SRQ mask at $C04F - the mask set by the `Q` command - and if the user has enabled service requests on input data errors it loads status code `$18` and asserts SRQ. If that bit is clear the character is discarded and the rest of the line is abandoned.

This confirms the Operation Manual's description of `Q2`, which gives an undecodable command as its example. It also means an unrecognised character silently truncates the remainder of the line even when SRQ is disabled, which the manual does not say.

6

Programming Reference

The Complete Command Set

The tables below are generated directly from the dispatch tables in the two ROM images and are therefore complete: a character not listed is not a command. The standard build recognises 23 characters, the CPR build 24.

Table 6-1. Standard Build Command Characters

Char	Function	Handler	In manuals
U	Step control: up	$ECA5	Yes
D	Step control: down; also the prefix of DON and DOFF	$ECAD	Yes
L	Step control: left	$ECE3	Yes
R	Step control: right; also the prefix of RM and RL	$ECE9	Yes
S	Prefix of SM, store to user memory	$ED25	**No**
A	Reset, equivalent to Device Clear	$F9CD	Yes
Q	Service request mask	$ED47	Yes
E	Output delimiter select	$ED5E	Yes
M	Fast / slow mode select	$ED75	Yes
T	Two / four-wire select	$ED8C	Yes
P	Parallel poll response select	$EDA3	Yes
N	Advance the calibration sequence	$EDBA	**No**
.	Decimal point in numeric entry	$EDCF	Yes
0 - 9	Numeric entry	$EDE8	Yes

Table 6-2. CPR Build Command Characters

Char	Function	Handler	In manuals
U	Step control: up	$EF4E	Yes
D	Step control: down; also the prefix of DON and DOFF	$EF56	Yes
L	Step control: left	$EF8C	Yes
R	Step control: right; also the prefix of RM and RL	$EF92	Yes
S	Prefix of SM, store to user memory	$EFCE	**No**
A	Reset, equivalent to Device Clear	$FD46	Yes
Q	Service request mask	$EFF0	Yes
E	Output delimiter select	$F007	Yes
C	CPR / active mode select (CPR build only)	$F035	Yes
M	Fast / slow mode select	$F01E	Yes
T	Two / four-wire select	$F04C	Yes
P	Parallel poll response select	$F063	Yes
N	Advance the calibration sequence	$F07A	**No**
.	Decimal point in numeric entry	$F08F	Yes
0 - 9	Numeric entry	$F0A8	Yes

Note



Command characters are matched as upper case only. There is no case folding anywhere in the receive or dispatch path, so m1 is not M1. The single exception is the exponent delimiter inside a number, where the numeric parser tests for both E and e, exactly as the Operation Manual states.

Argument Ranges Enforced by the Firmware

Each argument-taking command range-checks its digit and rejects the whole line if the digit falls outside. The limits below are read from the comparison instructions in each handler.

Table 6-3. Enforced Argument Ranges

Command	Accepted	Stored at	Manual states
Q<n>	0 - 7	$C04F	Q0 to Q7 - agrees
E<n>	0 - 4	$C050	E0 to E4 - agrees
P<n>	0 - 8	$C051	Range not stated
T<n>	0 - 1	$C052	T0, T1 - agrees
M<n>	0 - 1	$C053	M0, M1 - agrees
C<n>	0 - 1	$C054	C0, C1 - agrees
SM<n>	0 - 9	$C066	Command not documented
RM<n>	0 - 9	$C066	Command not documented

The P range of nine values is consistent with the manual's description - P0 selects no response and the remaining eight select one of the DIO lines - but the manual never states the upper limit. P9 is rejected.

Numeric Entry

A digit or a decimal point begins numeric entry, which then runs its own loop outside the dispatch table until a character it does not recognise is reached. That character is pushed back and the interpreter resumes. The accepted forms are digits, one decimal point, and an exponent introduced by `E` or `e` and optionally signed.

The exponent parser accumulates digits into a single byte at $C055 and records a minus sign at $C056. Note that it keeps only the last digit pair it forms, so exponents are not range-checked in the way the mantissa is.

Commands Absent from the Manuals

Four command forms exist in both builds and appear in neither the Operation Manual nor the Service Manual. All four have front-panel equivalents, which is almost certainly why they exist: the same handler serves both the key and the bus. Chapter 7 gives a full entry for each.

Table 6-4. Undocumented Command Forms

Form	Effect	Front-panel equivalent
`SM<n>`	Store the present value into user memory	`STO MEM` then a digit
`RM<n>`	Recall user memory into the present value	`RCL MEM` then a digit
`RL`	Recall the last entered value	`RCL LAST`
`N`	Advance the calibration sequence one step	The unit keys during calibration

Caution



`SM<n>` writes to non-volatile memory. `N` drives the calibration state machine and, at the end of the sequence, commits calibration constants to NOVRAM. Neither should be sent to a calibrated instrument casually. See Chapters 7 and 8.

7

Command Reference

Introduction

This chapter documents every command the firmware's dispatch table accepts, as a set of formal entries. Chapter 6 gives the command set as a whole and the ranges the firmware enforces; this chapter is the per-command reference.

The instrument has no subsystem structure - the interpreter dispatches on a single character - so entries appear in the order the dispatch table holds them, which is the order given in Table 7-1. Unless a difference is noted, every command applies to both builds.

Notation

Each entry opens with the command character, followed by one line for each accepted form. `<n>` is a single decimal digit; the permitted range is stated in the text and is enforced by the firmware. Characters shown literally are matched literally and are not case-folded.

An entry closes with a qualifier in parentheses. **(No response)** means the command produces no output on the bus - the 2724A is a source, and no command returns data. **(Not documented)** means the command works but appears in neither factory manual. **(CPR build only)** means the standard firmware has no dispatch entry for it and will treat it as an undecodable command.

Command Summary

Handler addresses are given for both builds where the command exists in both. These are the addresses the dispatch table branches to, computed from the 16-bit offsets the table stores.

Table 7-1. Dispatch Table Contents

Command	Standard handler	CPR handler	In the manuals
A	$F9CD	$FD46	Documented
D	$ECAD	$EF56	Documented
C	absent	$F035	Documented
E	$ED5E	$F007	Documented
L	$ECE3	$EF8C	Documented
M	$ED75	$F01E	Documented
N	$EDBA	$F07A	Not documented
P	$EDA3	$F063	Documented
Q	$ED47	$EFF0	Documented
R	$ECE9	$EF92	Bare form documented
S	$ED25	$EFCE	Not documented
T	$ED8C	$F04C	Documented
U	$ECA5	$EF4E	Documented
.	$EDCF	$F08F	Documented
0 - 9	$EDE8	$F0A8	Documented

Command Entries

A

A

Resets the instrument, equivalent to an IEEE-488 Device Clear. The handler is the reset entry point, so the full power-on sequence runs, including the ROM and RAM self-tests and the greeting.

The Operation Manual notes this is provided for controllers without a Device Clear capability. Handler $F9CD standard, $FD46 CPR. **(No response)**

C

C0

C1

`C0` selects active mode, the power-up condition. `C1` selects CPR mode. The digit is stored at `$C054`, the same byte the CPR front panel's mode key toggles - see Chapter 3.

The standard build's dispatch table has no `C` entry, so a standard instrument treats `C1` as an undecodable command and abandons the rest of the line. Handler $F035. **(CPR build only)**

D

`D`

`DON`

`DOFF`

Bare `D` is the step-control down arrow.

`DON` turns step control on and `DOFF` turns it off. The handler peeks at the next character without consuming it: if it is `O` the longer forms are parsed, otherwise the bare arrow action is taken.

Any other letter after `DO` is an error, as is `DOF` with a single F. Handler $ECAD standard, $EF56 CPR. **(No response)**

E

`E<n>`

Sets the output delimiter. The digit is range-checked to 0 through 4 and stored at `$C050`; a digit outside that range is an undecodable-command error.

Note that `E` is also the exponent delimiter inside a number. The two uses do not conflict because the numeric parser consumes the exponent itself and never returns to the dispatch table while a number is being read. Handler $ED5E standard, $F007 CPR. **(No response)**

L

`L`

Step-control left arrow. Moves the active digit one place towards the most significant end. Handler $ECE3 standard, $EF8C CPR. **(No response)**

M

`M0`

`M1`

`M0` selects slow mode, the power-up condition. `M1` selects fast mode. Range-checked to 0 or 1 and stored at `$C053`, from which bit 7 of the latch at $0E00 is derived. Handler $ED75 standard, $F01E CPR. **(No response)**

N

`N`

Advances the calibration sequence by one step, accepting whatever value has been entered since the previous step. It is the bus equivalent of pressing a unit key during the front-panel calibration procedure of Section 6.3.4; the three unit keys and this command share one handler, reached as key 11 of the key dispatch table.

The handler begins by testing the calibration step counter at `$C012` and returns immediately if it is zero. Outside calibration the command is therefore harmless and has no effect at all.

Within calibration it commits the entered value, increments the step counter and advances the sequence. When the counter reaches ten the sequence completes and the constants are written to NOVRAM. Handler $EDBA standard, $F07A CPR, both reaching $F30E / $F5EF.

Caution



Sending `N` to an instrument whose rear calibration keyswitch is enabled will advance a calibration in progress and can commit wrong constants to non-volatile memory. See Chapter 9.

(Not documented)

P

`P<n>`

Sets the parallel poll response. `P0` selects no response; `P1` through `P8` select the DIO line the instrument asserts. The digit is range-checked to 0 through 8 and stored at `$C051`.

The Operation Manual describes the command but does not state the upper limit. `P9` is rejected. Handler $EDA3 standard, $F063 CPR. **(No response)**

Q

`Q<n>`

Sets the service-request mask. The digit is range-checked to 0 through 7 and stored at `$C04F`, where it is read as three independent enable bits: bit 0 internal errors, bit 1 input data errors, bit 2 settled to a new value.

The firmware's range agrees exactly with the eight combinations the Operation Manual documents as `Q0` through `Q7`. Handler $ED47 standard, $EFF0 CPR. **(No response)**

R

R

RM<n>

RL

Bare R is the step-control right arrow.

RM<n> recalls user memory , where is 0 through 9, into the present value. Before overwriting, the present value is exchanged with the last-value buffer at $C1CA, so a recall can itself be undone with RL.

RL recalls the last entered value, exchanging the present value with the last-value buffer. It is the bus equivalent of the front-panel RCL LAST key, and is refused while a calibration or an entry is in progress.

The handler peeks at the character after R without consuming it, so a bare R followed by anything that is not M or L is simply the arrow. Handler $ECE9 standard, $EF92 CPR. **(Bare form documented; RM and RL are not)**

S

SM<n>

Stores the present value into user memory , where is 0 through 9. The eight bytes at $C069 are copied to $C35C plus eight times , the user block checksum is recomputed, and the whole non-volatile image is written to NOVRAM.

It is the bus equivalent of the front-panel STO MEM key followed by a digit.

S is not a command on its own. The handler requires the next character to be M and treats anything else as an undecodable command. Handler $ED25 standard, $EFCE CPR.

Caution



Every SM<n> triggers a NOVRAM store cycle. These devices tolerate a finite number of such cycles; do not send this command in a polling loop.

(Not documented)

T

T0

T1

T0 selects four-wire correction, the power-up condition. T1 selects two-wire. Range-checked to 0 or 1 and stored at $C052. Handler $ED8C standard, $F04C CPR. **(No response)**

U

U

Step-control up arrow. Increments the active digit. Handler $ECA5 standard, $EF4E CPR. **(No response)**

Digits and decimal point

`0 - 9`

`.`

Any digit, or a decimal point, begins numeric entry. The parser then runs its own loop, accepting digits and one decimal point, until it meets a character it does not recognise; that character is pushed back and the dispatch table resumes.

An exponent is introduced by `E` or `e` and may be signed. Within the exponent, `+` and `-` are significant; at the top level of a command line they are discarded.

The Operation Manual's statement that a maximum of seven digits is accepted is a property of the value representation rather than the parser, which does not itself count digits. Handler $EDE8 / $EDCF standard, $F0A8 / $F08F CPR. **(No response)**

8

Non-Volatile Data

NOVRAM Organisation

IC522 and IC523 are 256 x 4 non-volatile RAMs. The Service Manual describes them as holding calibration data and user data respectively, and says that IC522's write control is switched by the rear calibration keyswitch while IC523's is under processor control. The firmware shows how the two devices share one address space.

Both devices occupy $A000 onward. IC522 supplies data bits 0 to 3 and IC523 supplies bits 4 to 7, so each address holds one nibble of calibration data and one nibble of user data. The firmware never treats an $A000 location as a whole byte of one kind; it always packs and unpacks nibbles.

Note



This arrangement explains a detail the Service Manual leaves implicit. Because the two blocks are interleaved by nibble rather than separated by address, the calibration keyswitch can protect calibration data while leaving user memories writable, without any address decoding to distinguish them.

Block Layout and Checksums

Two logical blocks are mirrored between RAM and NOVRAM. Each carries its own 16-bit additive checksum, stored immediately after the data it covers.

Table 8-1. Non-Volatile Blocks

Block	RAM extent	Bytes	Checksum	Covers	NOVRAM nibble
Calibration	$C300 - $C357	88	$C358 - $C359	$C300 - $C357	Low, IC522
User	$C35A - $C3AB	82	$C3AC - $C3AD	$C35A - $C3AB	High, IC523

The user block begins with two bytes of configuration at $C35A, which include the IEEE-488 address, and continues with ten user memories of eight bytes each from $C35C to $C3AB. The calibration block is 88 bytes of constants whose internal structure was not determined; it is written as an undifferentiated run by the store routine.

Both checksums are simple 16-bit sums of the covered range, formed by adding successive 16-bit words with `ADDD ,X++` and discarding carry out of bit 15.

Store and Recall

Transfer in each direction is a single routine. Storing, at $E0FA in both builds, recomputes both checksums, writes them into RAM, and then copies 170 bytes of RAM into 180 NOVRAM locations as nibble pairs. The final act is a read of $2000, which is not a data access - the address strobes the NOVRAM's store cycle, committing the shadow RAM inside the devices to their non-volatile array.

Recall, at $E165 in the standard build, reads $4000 to strobe the array recall, waits, reassembles both blocks from their nibbles and then verifies both checksums.

Table 8-2. NOVRAM Control Strobes

Address	Direction	Effect
$2000	Read	STORE - copy the device shadow RAM into non-volatile array
$4000	Read	RECALL - copy the non-volatile array into shadow RAM

Caution



Non-volatile RAMs of this type tolerate a limited number of store cycles. The firmware issues a store every time a user memory is written, so a program that sends `SM<n>` in a loop will consume that endurance quickly.

Corruption Behaviour

Recall checks the calibration checksum first. If it fails, the firmware loads a default calibration block from ROM and reports `CAL DATA BAD` . If the user checksum fails, a separate path reports `MEMORY DATA BAD`. Both messages are documented in Table 3-2 of the Operation Manual; the recovery behaviour behind them is not.

The consequence worth knowing is that a calibration checksum failure is not merely reported. The stored constants are replaced with ROM defaults, and the instrument's calibration is gone rather than merely suspect.

9

Calibration and Integrity Checks

The Calibration Step Counter

Calibration is a state machine driven by one variable: the byte at $C012. It is zero whenever the instrument is not calibrating. Entering calibration sets it to one, and each accepted step increments it. The sequence is complete when the counter reaches ten, which the code tests explicitly:

```
LDA   <$12          ; calibration step
CMPA  #$0A          ; ten steps in the sequence
LBCC  done          ; finished - commit and exit
```

Almost every part of the firmware consults this byte. The `N` command does nothing at all unless it is non-zero; the reset path clears it; the display code uses it to decide whether to show calibration prompts.

A step may be passed over without entering a value. The code tests whether any digits were accumulated and, if none were, advances without storing - which is the behaviour the Operation Manual describes in Section 6.3.4, where a step may be skipped by pressing a unit key without adjusting the output.

Driving Calibration Remotely

The Operation Manual presents calibration as a front-panel procedure. Section 6.3.4 describes pressing keys; nothing in Section IV suggests the sequence can be driven over the bus. It can.

The `N` command reaches the same handler as the front-panel unit key - entry 11 of the key dispatch table and the `N` entry of the command dispatch table both branch to $F30E in the standard build. A controller can therefore send a value followed by `N` to accept each calibration step, exactly as an operator would key it in and press a unit key.

Warning



The rear-panel calibration keyswitch must be on for the sequence to start, so this cannot happen by accident on a sealed instrument. But once that switch is on, a controller can complete a full calibration with no operator at the front panel, and the final step commits the result to NOVRAM. An automated test system that sends stray `N` characters to a 2724A with its calibration switch enabled can destroy the calibration without any warning being displayed.

At the end of the sequence the firmware displays `TURN CAL SW OFF` and then waits, testing bit 6 of $C017 until the keyswitch is returned to normal, before jumping to the reset entry point.

The Banner Integrity Check

The firmware holds its model and version banner in more than one place and checks the copies against each other. In the standard build the check compares fourteen bytes at $F9BE, which is the banner as displayed at power-on, with a second copy at $F766:

```
LDS   #$C23F          ; reset the stack
LEAX  549,PCR         ; $F9BE - displayed banner
LEAY  -54,PCR         ; $F766 - reference copy
LDB   #$0E            ; fourteen bytes
LDA   ,X+
CMPA  ,Y+
BNE   fail            ; copies differ
DECB
BNE   loop
```

If the two copies differ the firmware displays a fifteen-character message which, decoded through the character table, reads `FUCK FUCK FUCK`. The same message is reachable from a second site in the calibration path. It is stored three times across the two builds, at $E405 and $F775 in the standard ROM and at $E53F and $FAE1 in the CPR ROM.

The mechanism is a tamper check rather than a fault indication. The banner distinguishes a standard instrument from a CPR one, and a plausible motive for the check is to make a standard ROM that has been patched to claim CPR capability announce the fact. Whatever the intent, the practical point for a technician is that this message means the ROM has been altered, not that the instrument has a hardware fault.

Note



The banner text itself is `2724A V-FB` in the standard build and `2724CPR V-FB` in the CPR build. Its meaning is not explained anywhere in the manuals, which quote it without comment as the second power-on display.

10

Firmware Differences

Common Regions

The two builds are not a common body with a patch. Only 1247 of 8192 bytes - 15.2 per cent - are identical at the same address, and the CPR build's code is shifted throughout. What the two share is a small number of contiguous library regions that happen to sit at the same addresses in both.

Table 10-1. Regions Identical in Both Builds

Range	Bytes	Contents
$E002 - $E0A8	167	ROM checksum word, character table at $E058, first display messages
$E0DE - $E14D	112	NOVRAM checksum and store routines
$E16E - $E1B8	75	NOVRAM recall routine
$E1BB - $E201	71	Calibration default block loader
$FE71 - $FFF1	385	Low-level utilities, delay routines and the vector table

That the NOVRAM routines are byte-identical and identically placed means the non-volatile data format is common to both builds. A calibration block from a standard instrument has the same layout as one from a CPR instrument, though not necessarily the same contents.

Functional Differences

The two extra key handlers in the CPR build correspond to the two extra front-panel keys the Operation Manual describes for that option, `CARD PNT` and the CPR mode indication. The extra command character `C` is documented in Section 4.4 of the Operation Manual as being available on CPR units only, and the firmware bears that out: the standard build has no `C` entry, so sending `C1` to a standard instrument is an undecodable-command error.

Table 10-2. Differences Visible in the Firmware

Feature	Standard	CPR
Command characters	23	24
Extra command	-	`C0` / `C1`, active or CPR mode
Front-panel key handlers	15	17
Banner text	`2724A V-FB`	`2724CPR V-FB`
Cardinal-point calibration prompts	None	`CPR 1Ko/ CAL`, `CPR 1Mo/ CAL` and others
Command dispatch table	$EEC8	$F188
Front-panel key table	$F0C1	$F385
Display routine	$E9E6	$EC55
Reset entry	$F9D8	$FD51

11

Verification Record

Confirmed by Computation

No instrument was available while this document was written. Nothing in it has been confirmed by operating a 2724A. What follows is what was checked, and how.

Table 11-1. Checks Performed

Check	Method	Result
Images are complete and uncorrupted	Firmware's own ROM test recomputed over both images	Both match the word at $E000
Load address is $E000	All eight vectors decoded and tested for range	7 of 7 live vectors point into ROM in both builds
Processor is a 6809	Parts list, plus 6809-only instructions and the FIRQ vector	Consistent
Disassembler is correct	68 hand-verified encodings covering every addressing mode	68 of 68 pass
Command tables are complete	Tables walked to their zero terminators	23 standard, 24 CPR
Message decoding is correct	Record stride determined empirically over four candidate strides	15 bytes; all records form clean centred text
Documented commands agree with firmware	Argument ranges compared against Sections 4.4 to 4.6	All documented ranges agree
Greeting strap decoding	Index arithmetic evaluated for all four strap combinations and compared with note 4 of drawing 2724-071	4 of 4 rows reproduced exactly

The checksum result deserves emphasis. Reproducing an 8K checksum exactly, from an algorithm read out of the code, is strong evidence that the images are intact, that the load address is right and that the disassembly of at least the reset path is correct. It does not validate any interpretation of what the code means.

Not Verified

The following are inferences from reading code. They are stated in this document as fact only where the code leaves no reasonable alternative; everything in this list would be settled in minutes on a bench.

- That `SM<n>`, `RM<n>` and `RL` behave over the bus as described. The handlers are shared with the front-panel keys and the argument checking is unambiguous, but no command has been sent to an instrument.
- That `N` advances calibration. The evidence is a shared handler address and a step counter compared against ten. The full ten-step sequence was not traced.
- The meaning of the 88 calibration bytes at $C300. Their extent and checksum are certain; their internal structure was not determined.
- That $2000 and $4000 are NOVRAM store and recall strobes. This follows from the access pattern and from the device type in the parts list, but no timing or signal has been observed.
- The function of D703 and D704, and whether they drive bits 0 and 1 of $C017. Tracing K3 and K4 through to the keyboard buffer on drawing 2724-070 sheet 6 would settle it.
- The IEEE-488 controller type. The register layout is consistent with a conventional talker/listener controller, but the parts list for board 2724-602 does not name the device.

How to Reproduce

The analysis rests on two files and a 6809 disassembler. To repeat the checksum verification, which is the single most useful check, sum every 16-bit big-endian word from offset 2 to the end of an 8192-byte image modulo 65536 and compare with the word at offset 0.

```
s = 0
for a in range(2, 8192, 2):
    s = (s + (rom[a] << 8 | rom[a+1])) & 0xFFFF
assert s == (rom[0] << 8 | rom[1])
```

To find the command set in any similar build, locate the dispatch loop by its signature - a `CMPA ,Y+` followed by `LEAY 3,Y` and a `TST ,Y` - and read the program-counter-relative offset in the `LEAY` that precedes it. That offset points at the table.